Data structures and Algorithms

Name: DURGASREE AVVARU

Registered Mail: 2300031591cseh@gmail.com

Exercise 2:

```
package Data_structures_and_Algorithms;
import java.util.Arrays;
import java.util.Comparator;
public class Exercise_2 {
   static class Product {
      int productId;
      String productName;
      String category;
      public Product(int productId, String productName, String category) {
         this.productId = productId;
         this.productName = productName;
         this.category = category;
      }
      public String toString() {
         return "Product ID: " + productId +
               ", Name: " + productName +
               ", Category: " + category;
      }
   }
   public static Product linearSearch(Product[] products, int targetId) {
```

```
    for (Product product : products) {
        if (product.productId == targetId) {
            return product;
        }
    }
    return null;
}
public static Product binarySearch(Product[] products, int targetId) {
    int left = 0;
    int right = products.length - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (products[mid].productId == targetId) {
            return products[mid];
        } else if (products[mid].productId < targetId) {
            left = mid + 1;
        } else {
            right = mid - 1;
       }
    }
    return null;
}
public static void main(String[] args) {
    Product[] products = {
        new Product(105, "Laptop", "Electronics"),
        new Product(101, "Mobile", "Electronics"),
```

```
    new Product(104, "Shoes", "Fashion"),
    new Product(102, "Watch", "Accessories"),
    new Product(103, "Book", "Education")
};
int searchId = 103;
System.out.println("=== Linear Search ===");
Product result1 = linearSearch(products, searchId);
if (result1 != null) {
    System.out.println("Product Found: " + result1);
} else {
    System.out.println("Product Not Found");
}
Arrays.sort(products, Comparator.comparingInt(p -> p.productId))
System.out.println("\n=== Binary Search ===");
Product result2 = binarySearch(products, searchId);
if (result2 != null) {
    System.out.println("Product Found: " + result2);
} else {
    System.out.println("Product Not Found");
}
System.out.println("\n=== Analysis ===");
System.out.println("Linear Search Time Complexity:");
System.out.println("Best Case: O(1)");
System.out.println("Average Case: O(n)");
System.out.println("Worst Case: O(n)");
System.out.println("\nBinary Search Time Complexity:");
```

```
        System.out.println("Best Case: O(1)");
        System.out.println("Average Case: O(log n)");
        System.out.println("Worst Case: O(log n)");
        System.out.println("\nConclusion:");
        System.out.println("Binary Search is more efficient for large datasets");
        System.out.println("because it reduces the search space by half in each step.");
        System.out.println("However, the array must be sorted before performing Binary Search.");
    }
}
```

Exercise 7:

```
package Data_structures_and_Algorithms;

public class Exercise_7 {
    public static double predictFutureValue(double currentValue, double growthRate, int years) {
        if (years == 0) {
            return currentValue;
        }
        return predictFutureValue(currentValue * (1 + growthRate), growthRate, years - 1);
    }
    public static void main(String[] args) {
        double currentValue = 10000; // Initial investment
        double growthRate = 0.10;    // 10% annual growth
```

```
        int years = 5;
        double futureValue = predictFutureValue(currentValue, growthRate, years);
        System.out.println("=== Financial Forecasting ===");
        System.out.println("Current Value : Rs." + currentValue);
        System.out.println("Growth Rate   : " + (growthRate * 100) + "%");
        System.out.println("Years         : " + years);
        System.out.printf("Future Value  : Rs.%.2f%n", futureValue);
        System.out.println("\n=== Analysis ===");
        System.out.println("Recursion:");
        System.out.println("A recursive function calls itself until a base condition is met.");
        System.out.println("\nTime Complexity:");
        System.out.println("O(n), where n is the number of years.");
        System.out.println("\nOptimization:");
        System.out.println("1. Use Iteration instead of Recursion.");
        System.out.println("2. Use Memoization for complex recursive problems.");
        System.out.println("3. Direct Formula: Future Value = Present Value * (1 + rate)^years");
    }
}
```